

Adapting Agile Approaches to Your Project Needs

Stephen J. Mellor

To kick off the millennium, Kent Beck wrote *eXtreme Programming eXplained*, with all the appropriate capitalization designed to attract attention.¹ This book and the practices behind it have had, and continue to have, a significant impact on the industry,



**You can build
a system
without
processes
and tools, but
you can't build
a system
without people.**

but the book has also caused an extraordinary degree of vitriol. (For a relatively polite discussion of agile ideas with Terry Bollinger, Watts Humphrey, and others, see www.computer.org/software/dynabook.) The reasons appear to include a focus on writing programs rather than analysis or design (also known as modeling); a disdain for documentation as such; and the Communist notion of working only 40 hours a week.

XP is not the only “lightweight” method. Others include Alistair Cockburn’s Crystal family,² now merged with Jim Highsmith’s Adaptive Systems Development³ and Scrum⁴—both mentioned in articles in this issue. These methods and others are similar to XP, but sufficiently different to cause confusion in the marketplace. Accordingly, in late 2000, Robert Martin from Object Mentor proposed that the various proponents of lightweight methods meet to form a consortium that would promote their common ideas. This proposal came to fruition in February 2001 when 16 lightweight-method proponents and I met to form the Agile Alliance. (The label “extreme” was viewed as too, well, extreme, and “lightweight” was considered too, sorry, lightweight.) During the meeting, a manifesto was born and a set of principles declared.

I introduced myself at this first meeting as “a spy” because my own interest has always been executable models—the notion that a model can be precise and detailed enough to be executed. Consequently, the idea that code is the only product of interest and that models are superfluous disturbs me. Certainly, we should focus our attention on the final product, but that product can surely be an executable model. I hoped to derail their evil plans.

The Agile Manifesto

Notwithstanding my concerns, the result was the Agile Manifesto (www.agilemanifesto.org). It begins thus: “We are uncovering better ways of developing software by doing it and helping others do it.” There are then four statements of the form “We value: <left> over <right>.” The key point here is not that <right> is bad but that <left> deserves more emphasis than it often receives in plan-based processes.

Let’s look at the four value statements in turn.

Individuals and interactions over processes and tools

Processes and tools are not silver bullets. You can build a system without processes and tools, but you can’t build a system without people. And the better the people, the better the result. By far. None of this is to say that process and tools are unimportant.

Working software over comprehensive documentation

If you intend to drive from your home to the local hospital, you wouldn’t write a document that says exactly how to do it first. Certainly, you need a map that lays out the most effective way to get there, and you should listen to the radio for up-to-the-minute information on whether a particular route is blocked, but “documenting” the route for posterity is unlikely to stop the bleeding.

Customer collaboration over contract negotiation

We’ve all heard of projects where the specification was “frozen,” often with unfortunate results. A specification, even an informal one, is just a framework, so we should expect to work with our customers to ensure we build the right product.

Responding to change over following a plan

A plan tells us how to get from where we are to where we’re going, but it doesn’t tell us if where we’re going is the right place to be. Even if the plan is correct—today—this doesn’t mean it’s the right place tomorrow. Technology and the market change far too quickly for that. Achieving the plan milestones doesn’t necessarily equate to customer success.

This value statement relies on distinguishing between predictive and adaptive approaches. In the construction industry, executing a plan—the construction phase—takes relatively much longer than the planning and design phases. We use the plan to predict when the project will be complete. In software, the design and planning phase takes much longer than the execution. Consequently, the plan doesn’t predict much of value—we must adapt what we’re doing as circumstances change.⁵

The key here is to exploit change to ensure that the product is as close as possible to what the customer needs. Reality provides real feedback.

Key ideas

By now, you've probably considered the Manifesto and developed your own mental list of what's key for your work. Here are a few I think deserve highlighting.

Code vs. models

The value statements refer to “software,” not “code.” As such, contrary to common wisdom, they don't prohibit building models so long as the models are executable rather than blueprints to be filled out—eventually—with running code. In fact, most of the value statements, and the Manifesto's principles, apply to customer management, project management and environment, and early testing—not “code.” (Though I'll admit it was an interesting struggle to ensure we used the word “software” instead.)

Just enough

Agile methods are a welcome shake-up, but like many reactions to an entrenched position, it's possible to go too far. We must ask what is “just enough” process to meet the goal of, say, making requirements traceable while finishing the project as quickly as possible. This is especially important in long-lived or high-assurance systems, where fast-and-loose is deadly.

Teams

Pair programming is a variation on teams, about which much has been written. Organizations often say they support teamwork and even have inspirational posters around to reinforce the idea, but surprisingly little effort is made to keep successful teams together. Keeping three people on an analysis/modeling task doesn't reduce productivity by two-thirds, it increases quality.⁶

Adaptability

Agile methods emphasize adaptability. Consider three main areas:

- the market, which might demand different features today from those you're building;
- the technology, which makes new things possible by changing all the time but also sets limits that we might not discover until we're some way through the project; and
- the process itself.

Take as your mantra “Process check!” to ensure you're doing what's needed in the best possible way.

Documents and documentation

I believe the agile folks are right that many documents simply restate, poorly and imprecisely, what's better said in “the real thing.”

My main concern is that an oral tradition where we all sit around the fire and recount the glory of our project days just doesn't cut it for most systems. Executable models aren't enough either. Rather, we need to emphasize describing and elaborating the decisions we've used in the course of development. We shouldn't document the models but why we selected a particular abstraction and—most especially—why not another.⁷

The agile processes rely on customer interaction to tell us which features (stories, use cases, whatever you call them) to implement next. In many systems, the user-visible requirements are few. The customer does not supply the infrastructure requirements directly; they are derived. We need some approach to prioritize these requirements in the absence of a customer.

The articles

Because I value all these principles, I was pleased when I was asked to write this introduction. The four articles on this topic in this issue cover four different interest areas within the agile arena. “Project Management in Plan-Based and Agile Companies” by Martina Ceschi, Alberto Sillitti, Giancarlo Succi, and Stefano De Panfilis discusses the concerns of a range of project managers from both the agile and plan-based camps. The article illuminates real project concerns and describes how agility can help. It acts as a guideline for choosing to move (or not) in an agile direction.

The second article, “Context-Adaptive Agility: Managing Complexity and Uncertainty” by Todd Little, describes an approach for categorizing projects according to specific profiles that serve as a basis for adapting the agile process. The article helps you determine how best to adapt an agile process to suit your project's needs.

The third article, “Primavera Gets Agile: A Successful Transition to Agile Development” by Bob Schatz and Ibrahim Abdelshafi, is a case study on introducing Scrum into a particular company. It explains what really happened on a real project when the company adopted agile practices—to build a project management tool, no less. The lessons learned are especially valuable.

Agile methods are a welcome shake-up, but like many reactions to an entrenched position, it's possible to go too far.

About the Author



Stephen J. Mellor is chief scientist of the Embedded Systems Division at Mentor Graphics. He is active in the Object Management Group, chairing the consortium that added executable actions to the UML, and in specifying MDA. He is also a signatory to the Agile Manifesto and chair of *IEEE Software's* Advisory Board. Contact him at stephen_mellor@mentor.com; www.projtech.com.

The final article, "Combining Agile Methods with Stage-Gate Project Management: Lessons Learned" by Daniel Karlström and Per Runeson, examines three case studies of agile processes in global companies. The authors list high-importance agile practices and their effects—good and bad. These lessons learned should be helpful in your decision to implement agile practices and in successful implementation of them.

I attended that first agile meeting as a spy. I was astonished to see how many agile practices applied equally to executable

modeling. I came to derail their evil plans to deprecate modeling and, with a few tweaks (and reservations, it must be said), I was able to become a signatory to the Manifesto.

If you find these practices helpful, you might just kick off your own new millennium. ☺

References

1. K. Beck, *eXtreme Programming eXplained: Embrace Change*, Addison-Wesley, 2000.
2. A. Cockburn, *Agile Software Development*, Addison-Wesley, 2002.
3. K. Schwaber, *Agile Project Management with Scrum*, Microsoft Academic, 2004.
4. J.A. Highsmith, *Adaptive Software Development: A Collaborative Approach to Managing Complex Systems*, Dorset House, 2002.
5. M. Fowler, *The New Methodology*, Apr. 2003; www.martinfowler.com/articles/newMethodology.html.
6. L. Williams et al., "Strengthening the Case for Pair Programming," *IEEE Software*, July/Aug. 2000, pp. 19–25; www.cs.utah.edu/~lwilliam/Papers/ieeeSoftware.PDF.
7. L. Starr, *Executable UML: How to Build Class Models*, Prentice Hall, 2002.

**CALL
FOR
ARTICLES**

PUBLICATION DATE: March/April 2006
SUBMISSION DEADLINE: 1 Sept. 2005

State of the Practice & Future Directions for Software Architecture

SOFTWARE ARCHITECTURE captures and preserves designers' intentions about system structure and behavior, thereby providing a defense against design decay as a system ages. This issue's articles will emphasize techniques, tools, and software engineering principles that support organizations taking an architecture-centric approach to software development. We especially seek articles that offer practical techniques and guidance to help organizations improve their software architecture practice, articles that describe practitioner views of research challenges for the next 10 years, and survey articles by practitioners and researchers expressing the progress made over the last 10 years.

POSSIBLE TOPICS INCLUDE BUT ARE NOT LIMITED TO

- Architecture-based development
- Architecture governance
- Economic and business issues
- Software quality
- Evaluation
- Documentation
- Education
- Design
- Modeling
- Recovery
- Evolution

For the complete call, go to www.computer.org/software/edcal.htm.

Guest editors:

Judith A. Stafford, Tufts University, USA;
jas@cs.tufts.edu
Henk Obbink, Philips Research Laboratory,
the Netherlands; henk.obbink@philips.com

For **author guidelines** and **submission details**, write to software@computer.org or go to www.computer.org/software/author.htm. **To submit**, go to <http://cs-ieee.manuscriptcentral.com/index.html>.